Revisão da arquitetura do banco: plano de correções (feedback QI Tech)

Contexto

Na apresentação da prévia da RFC, a QI Tech apontou que:

1. o banco está complexo demais;
2. os diagramas ER não batem com as tabelas reais;
3. existem colunas redundantes (começando por `suitability_question`);
4. `transaction` tem três colunas de conta (`requester_account_id`, `origin_account_id`, `destination_account_id`);
5. a idempotência usa um `request_hash` que não deveria existir.

Este documento traz o diagnóstico completo e as correções, em fases pequenas. Ao fim de cada fase rodamos `pytest -v` e paramos para revisar.

Decisões já tomadas:

- Enumeradores viram **tipo ENUM nativo do Postgres** (`CREATE TYPE ... AS ENUM`), no banco inteiro.
- A `Idempotency-Key` passa a ser **UNIQUE de verdade** (global), sem `request_hash`.
- A tarifa passa a **entrar numa conta do banco** (partidas dobradas na tarifa).

Respostas às perguntas da banca

Como geramos a chave de idempotência hoje (e o que está errado)

- **Não geramos.** Quem gera é o cliente da API (um `uuid4` por pedido, como os testes fazem) e manda no cabeçalho `Idempotency-Key`. O servidor só guarda.
- A única exceção é a liquidação no vencimento: `uuid5("maturity:{order_key}")` (`src/controllers/investment_order_controller.py`, `settle_maturities`).
- O problema: a chave **não é UNIQUE**, é única *por dono*: `UNIQUE(requester_account_id, idempotency_key)` em `transaction` e `UNIQUE(customer_id, idempotency_key)` em `investment_order`. Como a mesma chave pode existir para donos diferentes, precisamos do `request_hash` para saber "é o mesmo pedido?".
- **Bug real encontrado:** a ordem de investimento grava a mesma chave em dois lugares (na ordem e na `transaction` dela). Se o cliente usa a chave K num depósito e depois numa aplicação, a busca na `investment_order` não acha nada, o INSERT na `transaction` bate no UNIQUE e a API devolve **500**.
- Correção: a chave mora **num lugar só** (`transaction.idempotency_key UUID UNIQUE`, nula nas transações que o banco cria sozinho). A ordem acha a sua chave pela transação dela (relação 1:1).
  - Chave repetida pelo mesmo dono na mesma rota: devolve a resposta original (200).
  - Chave já usada por outro dono ou noutra rota: 409 `PPP000004`, sem mostrar nada do outro (R8).
  - Sem hash.

Por que as três colunas de conta em `transaction` estão erradas

`ledger_entry` já diz exatamente quais contas foram afetadas. Na `transaction` (o *pedido*), `requester` sempre repete `origin` ou `destination`, e no depósito a "origem" nem existe. Fica:

- `account_id NOT NULL`: a conta que fez o pedido (dona da transação);
- `destination_account_id`: só na transferência (`CHECK` amarrando ao tipo `TRANSFER`).

O comprovante e a contraparte do extrato saem disso e do tipo; a resposta da API não muda.

Como reportamos o capital do cliente

Hoje o capital aparece em dois pedaços que ninguém junta:

- `GET /customer/{ck}/account/{ak}` devolve `balance`: só o dinheiro em conta (caixa);
- `GET /customer/{ck}/positions` devolve `total.current_amount`: os investimentos ao preço do dia.

Ninguém responde "quanto o cliente tem no banco" nem "quanto disso está em caixa e quanto em investimentos".

Descartado: guardar o total numa coluna (`balance` = patrimônio, mais duas colunas de percentual). O valor dos investimentos muda todo dia com o preço, então a coluna ficaria velha a cada fechamento e o fechamento teria que reescrever todas as contas. Além disso, `balance` hoje é exatamente a soma do extrato (créditos − débitos) e é a linha travada com `FOR UPDATE` para decidir saque e aplicação. Misturar investimento nele quebra esse invariante.

Escolhido: calcular na leitura, na rota de posições, que já busca o preço do dia de cada posição. Custo: ler a conta não encerrada do cliente e fazer uma soma. Sem coluna nova, sem escrita. O `total` da resposta ganha:

Campo	Significado
`cash_amount`	`account.balance` da conta não encerrada (0 se não houver)
`current_amount`	(já existe) investimentos ao preço do dia
`net_worth_amount`	`cash_amount + current_amount`: o capital total no banco
`cash_bps`	parte em caixa, em pontos-base (`10000` = 100%)
`investments_bps`	`10000 − cash_bps`, para a soma fechar sempre em 100%

Percentual em pontos-base inteiros, como o `return_bps` que já existe (sem `float`, R6). Patrimônio 0 → os dois percentuais são 0.

A movimentação do dinheiro está correta?

O núcleo está certo: saldo em centavos `BIGINT` com `CHECK >= 0`, `SELECT ... FOR UPDATE` em ordem de id, todo movimento vira `ledger_entry` com `balance_after`, um único commit. Dois furos:

1. **A tarifa some.** Sai da conta do cliente e não entra em lugar nenhum. Correção: uma conta do banco recebe o crédito `FEE`. A conservação do dinheiro fica mais simples: Σ saldos (com o banco) = depósitos − saques − aplicações + resgates.
2. **O bug de idempotência acima** (500 em vez de resposta idempotente ou 409).

Custo aceito (vai para a RFC): toda transferência trava a conta do banco, então as transferências passam a ser serializadas nela. Se isso virar gargalo, a saída é acumular a tarifa e creditar no fechamento do dia.

Diagnóstico completo de redundâncias

Tabela	Coluna(s)	Problema	Correção
`suitability_question`	`code` ('Q1') e `position` (1)	Mesma informação duas vezes	Tirar `position`; ordenar por `code`
`suitability_option`	`code` e `points`	Sempre iguais (o seed faz `points = code`). E "os pontos nunca saem na API" é falso: o `code` que sai *é* o ponto	Tirar `points`; pontuação = `code` (documentar)
`suitability_answer`	`question_id`	Sai de `option_id` (a opção pertence a uma pergunta)	Tirar; o controller já valida uma resposta por pergunta
`suitability_assessment`	`valid_until`	É `created_at + 24 meses`	Tirar; calcular no controller/DTO

Tabela	Coluna(s)	Problema	Correção
`suitability_assessment`	`forced_by_rule`	É "perfil gravado ≠ perfil da faixa do `score`"	Tirar; calcular no DTO
`model_portfolio`	tabela inteira	1:1 com `investor_profile`	Juntar: `name` vai para `investor_profile`; `model_portfolio_item.profile_id`
`transaction`	`requester/origin/destination`	Ver acima	`account_id` + `destination_account_id`
`transaction` e `investment_order`	`idempotency_key` + `request_hash` duplicados	Ver acima	Chave só em `transaction`, `UUID UNIQUE`; sem hash
`ledger_entry`	`entry_key`	Nunca sai na API (R5 só exige key no que sai)	Tirar
`investment_order`	`customer_id`	Sai de `account_id`	Tirar; os repositórios fazem join com `account`
`investment_order`	`source`	`MATURITY` ⇔ `settled_order_id` não nulo	Tirar; o DTO calcula (o campo `source` continua na API)
`investment_order`	`status`	Só aceita `EXECUTED`	Tirar
`product`	`own_issue`	⇔ `external_code IS NULL`	Tirar; derivar (ajustar fechamento do dia e RN-REC-06)
`account_status`, `transaction_type`, `product_type`	tabelas	Enumerador em tabela + `status_id <> 3` "mágico", enquanto `side` / `direction` / `indexer` são texto	Tipos ENUM nativos
`side`, `direction`, `category`, `actor`, `indexer`, `liquidity`, `reference_rate.code`	`VARCHAR + CHECK`	Padrão diferente do de cima	Tipos ENUM nativos (um padrão só)

Mantidos de propósito (cache justificado, explicar na RFC):

- `account.balance` e `position.quantity` / `position.invested_amount`: são a linha travada com `FOR UPDATE`; o invariante com o ledger e as ordens é testado.
- `ledger_entry.balance_after`: usado no extrato e na resposta idempotente.
- `investment_order.cost_amount`: necessário no resgate (custo médio).

Resultado: 21 → 17 tabelas, cerca de 14 colunas a menos, um padrão só de enumerador e um lugar só para a chave de idempotência.

Fases

Toda fase que mexe em `database/database.sql` exige `docker compose down -v` e `docker compose up -d --build`. TDD: teste black box primeiro (R1), ver falhar, implementar. A resposta da API não muda, salvo onde está dito.

Fase 1: idempotência (corrige o bug do 500)

- Testes novos em `tests/integration/test_transaction.py` e `test_investment_order.py`:
  - chave de um depósito reusada numa aplicação → 409 (hoje dá 500);
  - chave do cliente A usada pelo B → 409, sem dados de A;
  - mesma chave + corpo diferente → 200 com a resposta original, sem debitar de novo (substitui o teste atual "corpo diferente dá 409").

- `database.sql`: `transaction.idempotency_key UUID UNIQUE` (aceita nulo); tirar `request_hash` de `transaction` e `investment_order`; tirar `idempotency_key` de `investment_order`.
- `transaction_controller.py`: apagar `hash_request`; busca por chave global; dono ou rota diferente → `IdempotencyKeyReused`. A checagem de tamanho vira validação de UUID (400 `PPP000003`).
- `investment_order_controller.py`: acha a ordem pela transação; a liquidação grava chave nula (o `UNIQUE(settled_order_id)` continua impedindo pagar duas vezes); sai o `uuid5`.
- Repositórios e modelos de `transaction` e `investment_order`.

Fase 2: colunas de conta em `transaction` + conta do banco

- Testes: a transferência credita a tarifa na conta do banco; a conservação do dinheiro fecha.
- `database.sql`: `account_id` + `destination_account_id` com `CHECK`; seed da conta do banco (`account.customer_id` nulo só nela; o R8 nunca a alcança porque toda rota filtra por cliente).
- `transaction_controller.transfer`: 4 lançamentos (débito do principal, débito da tarifa, crédito do principal, crédito da tarifa no banco), travando a conta do banco junto, em ordem crescente de id.
- Ajustar `transaction_repository.get_by_key_for_account`, `transaction_dto` (contraparte e comprovante) e `investment_order_controller.money_transaction`.
- RFC: atualizar RN-REC-02/03 e a decisão "não existe conta de receitas" (R7: registrar a troca).

Fase 3: enumeradores como ENUM nativo

- `database.sql`: `CREATE TYPE` para cada enumerador; apagar `account_status`, `transaction_type` e `product_type`; o índice parcial vira `WHERE status <> 'CLOSED'`.
- Modelos: `sqlalchemy.Enum(..., name=..., create_type=False)` (o SQLAlchemy já está instalado).
- Controllers e DTOs: `x.status.enumerator` vira `x.status`; somem `TransactionRepository.get_type` e afins.
- Seeds de produto e carteira deixam de fazer join com tabela de tipo.

Fase 4: colunas redundantes restantes

Uma subfase por módulo, na ordem do projeto:

1. movimentações (`entry_key`);
2. suitability (`position`, `points`, `answer.question_id`, `valid_until`, `forced_by_rule`);
3. produtos (`own_issue`; atualizar `docs/precificacao-e-fechamento-do-dia.md`);
4. carteira (`model_portfolio` some);
5. ordens (`customer_id`, `source`, `status`).

Os campos que saem na API (`source`, `forced_by_rule`, `valid_until`) continuam, calculados no DTO ou no controller; os testes existentes provam que nada mudou para fora.

Fase 5: capital total do cliente na rota de posições

- Testes em `tests/integration/test_investment_order.py`: cliente só com caixa (100% caixa); depois de aplicar, `net_worth_amount = cash_amount + current_amount` e `cash_bps + investments_bps = 10000`; cliente sem nada (tudo 0).
- `investment_order_controller.positions`: busca a conta não encerrada (`AccountRepository.get_not_closed_by_customer`, que já existe) e passa o saldo ao DTO.
- `investment_order_dto.total_to_dict`: soma e percentuais. Nenhuma mudança no banco.
- RFC: RN-POS (novos campos) e a decisão "calcular na leitura × guardar coluna" (R7).

Fase 6: documentação sem divergência

- `docs/diagramas/er-*.mmd`: **todas** as colunas de cada tabela, iguais ao `database.sql`. Hoje faltam, por exemplo, `transaction.description`, `suitability_question.text/position`, `product.name/issuer/indexer/rate_description/own_issue`, `investment_order.status` e vários `created_at`. Regerar os PNG com `npx -p @mermaid-js/mermaid-cli mmdc`.

- Script pequeno `scripts/check_er.py` (só biblioteca padrão): lê os `CREATE TABLE` do SQL e os blocos dos `.mmd` e lista as diferenças. Roda à mão depois de mexer no SQL.
- `docs/regras_de_negocio.tex`: tabela de colunas, RN-GER-08, RN-ORD-15, RN-FEC-09, catálogo `PPP000004` ("chave já usada por outro pedido") e decisões descartadas (R7: hash, chave por dono, tabelas de enumerador, receita sem conta). Apagar o diagrama mermaid duplicado do apêndice e apontar para os `.mmd`.
- `docs/rfc_fluxos.tex` e `docs/bootcamp-rfc-v0.md`: passos de idempotência e de transferência.

Verificação

- Fim de cada fase: `docker compose down -v`, `docker compose up -d --build`, esperar a API ficar healthy e rodar `pytest -v` (tudo verde).
- Fase 1: o teste "chave do depósito reusada na aplicação" falha com 500 antes e passa depois.
- Fase 2: soma dos saldos (com o banco) = depósitos − saques − aplicações + resgates.
- Fase 5: `GET /positions` de um cliente com caixa e aplicação mostra o total e os percentuais.
- Fase 6: `python scripts/check_er.py` sem diferenças; PNGs abertos e conferidos.
- `grep -rn "request_hash\|requester_account_id\|origin_account_id\|own_issue\|status_id" src docs` sem sobras.